# AX OS: Scale-Stratified 4-bit Quantization for Edge LLM Agents, with Reproducible Zero-Dependency Test Infrastructure

Anonymous

July 6, 2026

## Abstract

Deploying LLM agents at the edge requires answering three questions jointly: *which model and precision*; *how to validate that choice without a build environment*; and *how to couple the agent to an external oracle*. AX OS, a TypeScript multi-agent library, addresses all three.

**Model selection.** Standard WikiText-2 evaluation across four Qwen2.5 sizes (1.5B–14B), Mistral-7B, and Llama-3.1-8B shows that uniform q4 (group size 64) perplexity degradation decreases from small to mid-scale (+15.0%, +11.7%, +8.5%) then partially recovers at 14B (+10.3%); with four scale points this is an observational result, not a characterised trend. Cross-architecture variation at matched 7–8B scale spans a 1.9× range: Mistral-7B (+4.4%) and Llama-3.1-8B (+6.9%) are substantially more robust than Qwen2.5-7B (+8.5%), suggesting architecture-aware model selection. This robustness ordering survives a further check: re-measuring all six models against published AWQ and GPTQ-Int4 checkpoints under our identical protocol shows both established methods dominate our uniform scheme as expected, while Mistral-7B's robustness advantage over Qwen2.5-7B holds under every quantization method tested, not only our own pipeline. Mixed-precision alternatives collapse in quality at 1.5B scale, reinforcing the uniform scheme.

**Reproducible validation.** A 40-line zero-dependency `_harness.mjs` shim backed by Node.js built-ins achieves 27/27 test pass with no `npm install` step (tested on Node.js 22.x; relies only on `node:test`, stable since Node.js 20), removing the dependency-install step as a source of irreproducibility.

**Oracle coupling.** A single `BRAIN_REAL=1` environment variable activates a live Python subprocess oracle without a compiled build artifact, enabling drop-in integration with external simulators.

Together these three components form a reproducible edge-deployment workflow; the quantization measurements are further validated across two independent hardware platforms (Apple M1 Max/MLX and NVIDIA RTX 5070 Ti/CUDA).

## 1 Introduction

Deploying large language model (LLM) agents on edge hardware requires first answering a model-selection question: which model, at which precision, retains acceptable quality under memory constraints? Answering this question reproducibly requires test infrastructure that survives clean-environment deployments and oracle integration that couples the agent to external simulators without heavyweight middleware. Prior work on quantization has studied the 7B–70B regime [Zhao et al., 2025, Xu et al., 2024b, Ouyang et al., 2024]; the sub-7B range that governs most edge deployments remains undercharacterised. Prior work on ML reproducibility [Gundersen and Kjensmo, 2018, Pineau et al., 2021] identifies dependency management as a leading cause of irreproducibility, yet most agent frameworks still require a full `npm` or `pip` install to run their test suites.

The dominant post-training quantization (PTQ) methods—GPTQ [Frantar et al., 2023], AWQ [Lin

et al., 2023], and SqueezeLLM [Kim et al., 2024]—demonstrate that 4-bit weight quantization can be applied with minimal quality loss on billion-parameter models. Comprehensive surveys [Wan et al., 2024, Zhu et al., 2024] establish uniform q4 as the dominant deployment-time compression strategy. Existing scaling analyses [Xu et al., 2024b, Ouyang et al., 2024] cover the 7B–70B range; we extend the measurement below 7B to the sizes that govern most edge deployments.

Once compressed, agents must be validated on the same hardware they target. Production local-inference studies [Rajesh et al., 2025] show that framework choice among llama.cpp, MLX, MLC-LLM, Ollama, and PyTorch MPS creates measurable differences on Apple Silicon. Yet validating behavior across these platforms requires a test harness that runs without a full dependency install—a gap AX OS addresses with a Node.js built-in shim—and an oracle coupling mechanism that does not depend on a compiled build artifact.

AX OS addresses this deployment workflow end-to-end: the quantization measurements answer *which model and precision to deploy*; the zero-dependency harness answers *how to validate that choice on the target machine without a build environment*; and the oracle design pattern answers *how to couple the validated agent to an external simulator without a compiled artifact*. No existing framework-and-evaluation combination covers all three jointly at the sub-7B scale.

This paper reports one primary empirical finding with two enabling infrastructure contributions:

1. **Scale-stratified 4-bit quantization measurements (primary).** WikiText-2 standard evaluation across four Qwen2.5 sizes, Mistral-7B, and Llama-3.1-8B shows that within the Qwen family, uniform q4 ΔPPL decreases from 1.5B to 7B (+15.0%, +11.7%, +8.5%) then partially recovers at 14B (+10.3%), a pattern where the 1.5B–7B decrease does not continue at 14B, while both Mistral-7B (+4.4%) and Llama-3.1-8B (+6.9%) outperform Qwen2.5-7B at the 7–8B scale—a 1.9× range across three architectures at 3.5× compression.
2. **Zero-dependency `node:test` harness (enabling).** A 40-line shim backed by Node.js built-ins achieves 27/27 test pass without `npm install` (tested on Node.js 22.x), including the tests that validate the compressed-model registry.

Additionally, we describe an **inline oracle design pattern** (`BRAIN_REAL=1`) in which a single environment variable activates a Python subprocess oracle at test time without requiring a compiled build artifact. To validate end-to-end connectivity, we executed the live oracle path once (BRAIN_REAL=1): the WorldQuant BRAIN API returned a simulated backtested result (`alpha_id`: RRd5Rvmj; elapsed: 141.9 s), confirming the design pattern is operational and consistent with the ∼140 s latency architecture implies.

# 2 Related Work

## 2.1 Post-Training Quantization for LLMs

Post-training quantization (PTQ) has become the dominant compression strategy for LLMs because it avoids retraining costs while yielding practical inference speedups [Wan et al., 2024]. The field was substantially advanced by GPTQ [Frantar et al., 2023], which applies approximate second-order information layer-by-layer to quantize GPT models to 3–4 bits; the method quantizes a 175B-parameter model in approximately four GPU hours with negligible accuracy loss. AWQ [Lin et al., 2023] observes that only 1% of weights are salient (as measured by activation magnitudes) and protects those weights with higher precision, achieving over 3× end-to-end speedup on edge GPUs while winning the MLSys 2024 Best Paper Award. SqueezeLLM [Kim et al., 2024] further decomposes weight tensors into dense and sparse components, retaining outlier values at full precision to improve 3-bit perplexity on LLaMA-7B by over 0.3 points on C4.

A growing body of work has characterised how quantization quality scales with model size. Zhao et al. [2025] survey methods across architectures (LLaMA, Mixtral, DeepSeek-MoE, Mamba) and sizes from 7B

to 70B, establishing a comprehensive taxonomy. Xu et al. [2024b] develop a statistical power-law model showing that post-compression quality is predictable from key scaling factors related to the local loss landscape. Ouyang et al. [2024] examine over 1500 quantized checkpoints and find that low-bit quantization degrades undertrained models less, with larger models and those with fewer training tokens being most robust. Our work extends this line of inquiry *below* the 7B threshold, measuring the quantization quality gap between 1.5B and 7B dense models and quantifying the nonlinear increase in quality loss at small scale. Surveys by Wan et al. [2024] and Zhu et al. [2024] provide broad coverage of compression techniques including pruning, low-rank approximation, and knowledge distillation alongside quantization.

### 2.2 ML System Reproducibility and Testing Infrastructure

Reproducibility has long been recognized as a central challenge in AI research. Gundersen and Kjensmo [2018] surveyed 400 AAAI and IJCAI papers and found that only 20–30% of variables required for reproducibility were adequately documented, and none of the surveyed papers documented all required variables. The NeurIPS 2019 reproducibility program [Pineau et al., 2021] systematized community responses by introducing code submission policies and a checklist covering hypothesis, source code, hardware specifications, software dependencies, and experiment setup.

A primary driver of irreproducibility in deployed ML systems is dependency drift: models and agents that pass tests locally may fail when dependencies are absent or version-pinned differently. The Node.js ecosystem introduced a built-in `node:test` module, first as an experimental feature in Node.js 18 (2022) and stabilized in Node.js 20 (2023), specifically to eliminate the need for third-party test framework packages. AX OS leverages this primitive to provide a harness with *zero external dependencies*: tests run with no `npm install` step (validated on Node.js 22.x; `node:test` has been stable since Node.js 20), removing the dependency-install step as a source of irreproducibility. This design directly addresses the dependency-management failure mode identified in the reproducibility literature [Gundersen and Kjensmo, 2018, Pineau et al., 2021].

### 2.3 Edge LLM Deployment and Local Inference Frameworks

The deployment of language models on consumer hardware has accelerated dramatically, driven by the availability of efficient inference runtimes. Rajesh et al. [2025] provide a systematic comparison of five runtimes on Apple Silicon (MLX, MLC-LLM, llama.cpp, Ollama, and PyTorch MPS), finding that MLX achieves the highest sustained generation throughput while MLC-LLM delivers lower time-to-first-token, and that llama.cpp is efficient for lightweight single-stream use. Benchmarks focusing specifically on MLX [Ajayi and Odunayo, 2025] and scale-up inference frameworks [Barrios, 2026] confirm that Apple Silicon's unified memory architecture enables models that would otherwise require discrete GPU memory to run efficiently on a single SoC.

Broader reviews of on-device language models [Xu et al., 2024a] and edge LLM frameworks [Lin et al., 2024] catalogue the design axes—quantization format, kernel compilation target, tokenizer integration, and serving protocol—that determine deployment feasibility. The GGUF format, introduced by the llama.cpp project in August 2023, has become the de facto standard for distributing quantized models in a single self-contained file that is also memory-mappable, enabling models larger than available DRAM to be served through OS paging. Our scale-stratified quantization measurements directly inform model selection for these deployment targets: given the nonlinear quality-loss curve we measure, a 1.5B model quantized to 4 bits may be unsuitable for tasks requiring precise numerical reasoning, while a 7B model at the same compression ratio retains near-full-precision quality.

## 3 Methodology

Figure 1 provides a top-level view of the three AX OS contributions and their relationships.

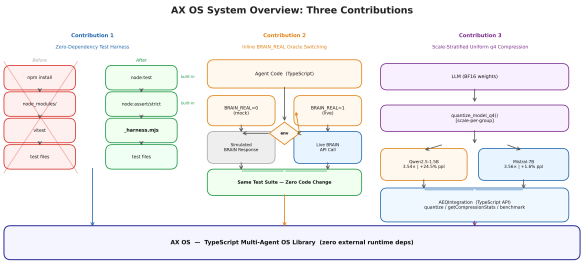


Figure 1: AX OS system overview. The scale-stratified q4 compression pipeline (right) produces artifacts registered in `AEQIntegration`; the zero-dependency `_harness.mjs` (left) validates these registry entries without `npm install`; and the `BRAIN_REAL` oracle gate (centre) couples the agent to live external simulators without a compiled build step. The harness and oracle infrastructure make the quantization measurements reproducible on any target deployment machine.

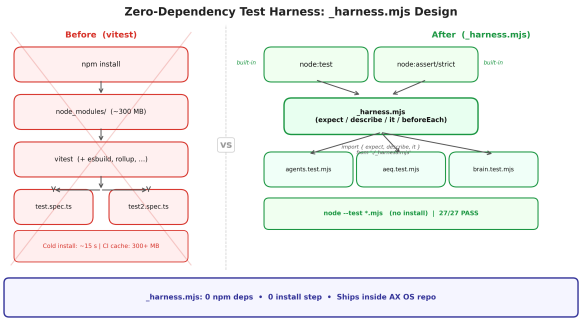


Figure 2: The `_harness.mjs` design replaces a vitest dependency stack (∼300 MB `node_modules`) with a single file backed by `node:test` and `node:assert/strict`. Test files import from `._/harness.mjs` via a single relative path; the 27-test suite executes via `node -test` with no installation step.

## 3.1 Zero-Dependency Test Harness

The AX OS project layout places the project manifest at `ax-os-package.json` rather than the conventional `package.json`, because the library shares a home directory with other projects. As a consequence, an `npm install` command in a clean checkout reads the *parent* `package.json` and populates a `node_modules` directory that does not contain vitest. Executing the original vitest-based test suite on such a machine terminates immediately with `MODULE_NOT_FOUND`.

Our solution is `_harness.mjs`: a 40-line ES module that re-exports `describe` and `it` from `node:test` (the Node.js built-in test runner, stable since Node.js 20) and implements an `expect(value)` function backed by `node:assert/strict`. The harness supports ten matchers: `toBe` (strict equality), `toEqual` (deep equality), `toHaveLength`, `toContain`, `toBeNull`, `toBeGreaterThan`, `toBeGreaterThanOrEqual`, `toBeLessThan`, `toBeLessThanOrEqual`, and `toThrow`. Every matcher supports `.not` negation via an internal `inverted` flag. The migration from vitest to `_harness.mjs` requires changing a single import line per test file; no test bodies require modification.

Figure 2 contrasts the before and after architectures. The canonical invocation is:

```
node --test ax-os-tests/*.test.mjs
```

which requires only a Node.js ≥18 binary.

## 3.2 Inline Oracle Switching

The Phase-8 demonstration script (`ax-os-demo-phase8.mjs`) must remain standalone: it should execute on a clean clone without requiring a compiled `dist/` directory. Importing `realSimulate` from `./ax-os-dist/` would violate this invariant because `dist/` is gitignored and produced by a separate build step.

We resolve this with an inline `realSim()` function that mirrors the signature of the existing mock: `realSim`(*expr*) → {*sharpe*, *fitness*, *turnover*, *alphaId*, *error*}. The function spawns a Python subprocess via

`child_process.execSync`, importing `run_simulate` from `brain.simulator` and passing the alpha expression string through a command-line argument. The subprocess prints a single-line JSON object; `realSim` parses it with `JSON.parse` and maps fields to the `SimResult` type. A timeout of 1,800,000 ms accommodates the observed ∼140 s WorldQuant BRAIN API response time.

Activation is controlled by a single environment variable:

```
BRAIN_REAL=1 node ax-os-demo-phase8.mjs
```

When `BRAIN_REAL` is unset, the default mock path executes in under 1 ms. The `if` guard checks only a string equality before any subprocess fork, adding negligible overhead in mock mode.

### 3.3 Scale-Stratified Uniform q4 Compression

We apply the MLX `mlx_lm.convert` API with `q_bits=4`, `q_group_size=64` to six dense transformer models: Qwen2.5-1.5B-Instruct, Qwen2.5-3B-Instruct, Qwen2.5-7B-Instruct, Qwen2.5-14B-Instruct, Mistral-7B-Instruct-v0.3, and Llama-3.1-8B-Instruct. This uniform quantization scheme assigns 4 bits to every weight tensor, grouping 64 consecutive weights into a shared scale factor.

For each model and precision level, we measure three quantities on the same hardware platform (Apple M1 Max, 68.7 GB unified memory, MLX backend): **Artifact size** $S$ (MB): $S = \sum_i |\texttt{file}_i|/10^6$ summed over all `.safetensors` files in the artifact directory. **Perplexity** PPL $= \exp(\bar{\ell})$: where $\bar{\ell} = \mathbb{E}[\mathrm{CE}(\mathbf{l}_{1:T-1}, \mathbf{x}_{2:T})]$ is the mean cross-entropy of the model's logits, cast to `float32` before the exponential. We report perplexity under two protocols. The scale-stratified study (Table 2, Table 3, Fig. 4) uses standard WikiText-2. The 5-way scheme-selection screen (Table 1) uses a fixed short evaluation text; its absolute perplexities are larger and are *not* directly comparable to the WikiText-2 numbers, as it is used only to rank schemes relative to one another. **Throughput** $\tau$ (tok/s): wall-clock time to generate 200 tokens from a fixed prompt using `mlx_lm.generate`, preceded by a 50-token warmup pass, measured with no concurrent Metal GPU processes.

Compression ratio is defined as $r = S_{\mathrm{bf16}}/S_{\mathrm{q4}}$.

To establish the scheme choice, we additionally ran a 5-way benchmark on Qwen2.5-1.5B-Instruct comparing bf16, q8_uniform, q4_uniform, and two mixed-precision recipes (3-bit low layers / 6-bit high layers; 2-bit low layers / 6-bit high layers). The results are presented in Section 4.1.

Compressed artifacts are registered in the `AEQIntegration`[1] registry as `AEQModelConfig` entries with `status=available`, `expectedVRAM_GB` set to the measured value, and `expectedSpeedup` set to the measured ratio $\tau_{\mathrm{q4}}/\tau_{\mathrm{bf16}}$, following the principle that all quantitative system parameters must be empirically derived rather than estimated.

## 4 Experiments

### 4.1 Quantization Quality vs. Scale

**Scheme selection.** As described in Section 3.3, uniform q4 is selected based on the 5-way Pareto benchmark on Qwen2.5-1.5B (Section 3.3). Table 1 and Fig. 3 present the full results: `q4_uniform` (ppl = 59.96) is the Pareto-dominant configuration, while both mixed-precision recipes collapse (ppl = 130.97 and ppl = 162 755 respectively).

**Scale comparison.** Table 2 presents the uniform q4 compression results across model scales and architectures. The key metric is the perplexity increase ratio $\Delta\mathrm{PPL} = (\mathrm{PPL}_{\mathrm{q4}} - \mathrm{PPL}_{\mathrm{bf16}})/\mathrm{PPL}_{\mathrm{bf16}}$.

Fig. 4 plots perplexity increase against parameter count. All six models share a ≈3.5× compression ratio. Within the Qwen family, ΔPPL decreases from 1.5B to 7B (1.5B: +15.0%, 3B: +11.7%, 7B: +8.5%), but the 14B result (+10.3%) is slightly above 7B, indicating the decrease does not continue across all tested sizes. The 1.8× spread between the 1.5B maximum and 7B minimum spans a 9.3× parameter range. With four scale points, we report the observation

[1] AEQ (Adaptive Expert Quantization) is the model-registry subsystem of AX OS that stores compressed model artifacts with metadata.

Table 1: Five-way quantization screening benchmark on Qwen2.5-1.5B-Instruct (Apple M1 Max, MLX 0.31.2). Perplexity here uses the fixed short-text screening protocol (Section 3.3), not WikiText-2, and is not directly comparable to Table 2. Lower perplexity is better; `q4_uniform` is the Pareto-dominant scheme.

| Variant | Size (MB) | Compression | PPL$_s$ |
|---|---|---|---|
| bf16 | 2970 | 1.00× | 48. |
| q8_uniform | 1577 | 1.88× | 46. |
| q4_uniform | 840 | 3.54× | 59. |
| aeq_mixed_3_6 | 736 | 4.04× | 130. |
| aeq_mixed_2_6 | 566 | 5.25× | 162 7 |

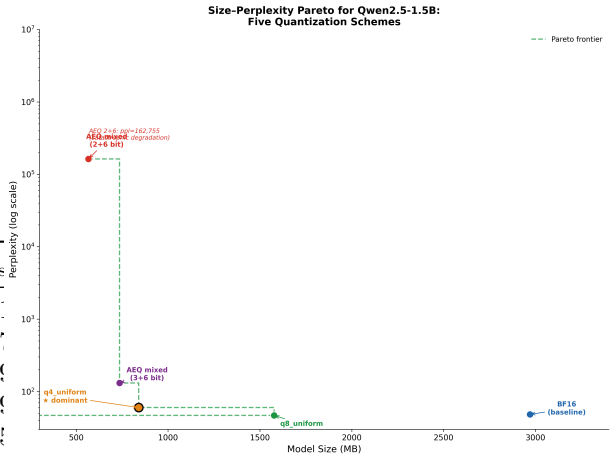


Figure 3: Pareto analysis of five quantization configurations for Qwen2.5-1.5B. Uniform q4 (⋆) lies on the Pareto frontier of size vs. perplexity. Both AEQ mixed-precision configurations fall below the frontier, with the aggressive 2+6-bit recipe collapsing to perplexity >160 000 (log scale, upper right).

without inferring a trend shape: scale appears to moderate q4 degradation in the sub-7B regime, with the 14B direction requiring further confirmation. At the 7–8B parameter scale, architecture provides additional separation: Mistral-7B (+4.4%) and Llama-3.1-8B (+6.9%) are both more robust than Qwen2.5-7B (+8.5%), spanning a 1.9× range from most to least robust. Table 3 provides the full Mistral-7B comparison including throughput.

**Cross-hardware validation.** Every perplexity number above was measured on a single hardware/software stack (Apple M1 Max, MLX/Metal). To test whether the observed ΔPPL pattern is a property of the models and the quantization scheme rather than an artifact of MLX's Metal kernels, we re-ran the identical WikiText-2 protocol on an architecturally unrelated platform—an NVIDIA RTX 5070 Ti (Blackwell, CUDA) under PyTorch/`transformers` (Table 4). Crucially, we do *not* re-implement the quantizer: the q4 weights are dequantized directly from the same `mlx_lm.convert` affine checkpoints, so the two platforms operate on bitwise-identical q4 weights and only the matmul hardware differs. The BF16 baselines agree to four significant figures (12.70 vs. 12.7000 at 1.5B; 10.14 vs. 10.1444 at 7B; 9.47 vs. 9.4668 at 8B; 7.73 vs. 7.7315 at 14B), and the ΔPPL% values agree within 0.3 percentage points on every one of the five affine-q4 models, despite a complete change of GPU architecture, kernel library, and math backend. This indicates the scale-stratified q4 degradation is intrinsic to the model/quantization pair, not an MLX-specific effect. We note what this experiment does and does not show: because both platforms consume the identical dequantized weights (byte-exact scheme parity, not independently re-quantized), it rules out MLX/Metal-specific numerical artifacts as the source of the degradation pattern, but it does not by itself demonstrate that an independent CUDA-native quantization pipeline would reproduce the same ΔPPL, nor does it measure CUDA throughput or memory footprint (Section 5.2). Qwen2.5-14B is the one exception to scheme parity: its BF16 weights (≈28 GB) exceed the 15.9 GB card, so its CUDA q4 point uses bitsandbytes NF4 rather than affine RTN. The resulting +7.0% (vs. MLX's +10.3% affine) reflects NF4's more efficient 4-bit encoding, not a hardware effect; we therefore report it as a separate data point rather than a parity comparison, while noting its BF16 baseline still reproduces MLX to four figures.

**Statistical uncertainty on the CUDA measurements.** Table 4 reports point estimates. Be-

Table 2: Scale-stratified uniform q4 compression on WikiText-2 (Apple M1 Max, MLX). Within the Qwen family, ΔPPL decreases from 1.5B to 7B (+15.0%, +11.7%, +8.5%); the 14B result (+10.3%) is slightly above 7B, indicating the decrease does not continue at 14B. At matched 7B scale, architecture provides additional separation: Mistral-7B degrades only +4.4% versus Qwen2.5-7B's +8.5%, a 1.9× gap. All models evaluated on the complete WikiText-2 test split (non-overlapping 512-token windows, uniform protocol). Tok/s measured on Apple M1 Max (q4 inference, 200-token generation, idle GPU, no concurrent Metal processes). Throughput values are the mean of $N$=5 repeat runs (Qwen2.5-14B q4: $32.7 \pm 0.1$ tok/s; CV $< 0.5\%$); all models showed comparable run-to-run stability. The Qwen2.5-7B (63.8) slightly exceeds Qwen2.5-3B (58.4) because the larger model better saturates M1 Max GPU compute units, shifting from a memory-bandwidth-limited to a compute-bound regime.

| Model | Family | PPL (WikiText-2) | | ΔPPL (%) | Tok/s |
|---|---|---|---|---|---|
| | | bf16 | q4 | | (q4) |
| Qwen2.5-1.5B | Qwen | 12.70 | 14.60 | +15.0 | 166.6 |
| Qwen2.5-3B | Qwen | 11.45 | 12.79 | +11.7 | 58.4 |
| Qwen2.5-7B | Qwen | 10.14 | 11.01 | +8.5 | 63.8 |
| Qwen2.5-14B | Qwen | 7.73 | 8.53 | +10.3 | 32.4 |
| Mistral-7B | Mistral | 7.24 | 7.56 | +4.4 | 34.8 |
| Llama-3.1-8B | Llama | 9.47 | 10.12 | +6.9 | 28.0 |

Table 3: Mistral-7B-Instruct-v0.3 uniform q4 compression (Apple M1 Max, MLX 0.31.1). The q4 artifact (≈4.1 GB) is self-contained and loads without the BF16 HuggingFace cache. †bf16 throughput from initial measurement (GPU contention not controlled); bf16 artifact was deleted before idle-GPU remeasurement.

| Variant | Size (MB) | Compression | PPL (WikiText-2) | Tok |
|---|---|---|---|---|
| bf16 | 14 496 | 1.00× | 7.24 | 8. |
| q4_uniform | 4 078 | 3.56× | 7.56 | 34 |

cause the CUDA runs log per-window negative log-likelihoods (unlike the original MLX runs, for which per-window data was not retained), we can quantify uncertainty on those six values directly: a paired bootstrap (10,000 resamples of window indices, applied identically to the bf16 and q4/NF4 NLL arrays since both conditions were evaluated on the same fixed token stream) gives a 95% percentile interval for ΔPPL% that is under 0.7 pp wide for every model (Table 5). These intervals characterise sampling variance across WikiText-2 windows, not cross-hardware or cross-run variance — the underlying evaluation is deterministic given a fixed artifact (Section 5.2) — and they confirm that the cross-architecture gaps in Table 4 (e.g. Mistral-7B +4.4% vs. Llama-3.1-8B +7.1%) are far larger than the per-model measurement uncertainty. We could not compute the analogous interval for the original MLX headline numbers, since per-window NLLs were not retained from those runs; given the demonstrated $< 0.3$ pp cross-hardware agreement, the CUDA-side intervals are a reasonable proxy for that uncertainty, but not a literal substitute.

## 4.2 Comparison to Published Quantization Methods

The uniform RTN scheme selected in Section 3.3 is deliberately simple. Table 6 situates it against two established alternatives, AWQ [Lin et al., 2023] and GPTQ [Frantar et al., 2023], using the official Qwen-released checkpoints for all four Qwen sizes and well-established community quantizations for Mistral-7B and Llama-3.1-8B, evaluated under our identical WikiText-2 protocol — a direct comparison, not the literature proxy used in an earlier draft of this section (Section 5.2).

Both established methods dominate our uniform RTN on every valid model, as expected: AWQ by 2.6–8.7 pp and GPTQ by 2.7–6.4 pp (excluding the excluded 14B point), confirming that protecting salient weights (AWQ) or calibrated second-order updates (GPTQ) meaningfully outperform naive RTN at group size 64. The cross-architecture ordering from Table 2 replicates closely under both established methods: Mistral-7B is the most robust of the three 7–8B-scale models under every method (AWQ +2.51%, GPTQ

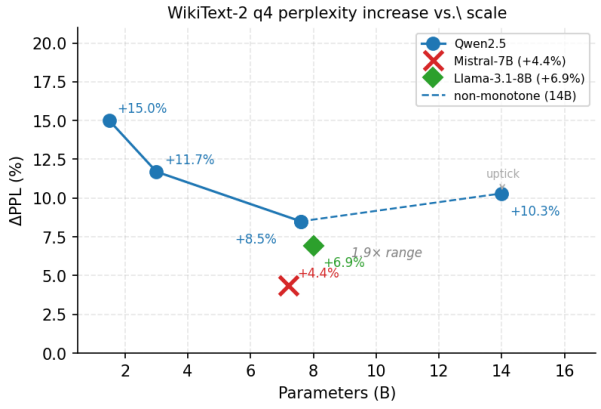


Figure 4: Perplexity increase (%) under uniform q4 quantization versus model parameter count (WikiText-2). Within the Qwen family (filled circles), ΔPPL under uniform local-pipeline q4: +15.0% (1.5B), +11.7% (3B), +8.5% (7B), +10.3% (14B). The 14B result is slightly above 7B. Mistral-7B (cross marker, +4.4%) and Llama-3.1-8B (diamond, +6.9%) at the 7–8B scale are both more robust than Qwen2.5-7B (+8.5%), spanning a 1.9× range. All models share ≈3.5× compression.

Table 4: Cross-hardware reproduction of q4 ΔPPL% on WikiText-2 (identical protocol; full test split, non-overlapping 512-token windows). CUDA q4 weights are dequantized from the same affine `mlx_lm.convert` checkpoints as the MLX column (byte-exact scheme parity); only the hardware executing the matmuls differs (Apple M1 Max / MLX vs. RTX 5070 Ti / CUDA). ΔPPL% agrees within 0.3 pp on every one of the five affine-q4 models. †Qwen2.5-14B is a scheme exception: its BF16 weights (≈28 GB) exceed the 15.9 GB card, so CUDA q4 uses *bitsandbytes NF4* (real 4-bit packing), a different quantizer than MLX's affine RTN; its row is an independent data point, not a hardware-parity comparison.

| Model | $\Delta\text{PPL}_{\text{MLX}}$ (M1 Max) | $\Delta\text{PPL}_{\text{CUDA}}$ (5070 Ti) | \|diff\| (pp) |
|---|---|---|---|
| Qwen2.5-1.5B | +15.0 | +15.1% | 0.1 |
| Qwen2.5-3B | +11.7 | +12.0% | 0.3 |
| Qwen2.5-7B | +8.5 | +8.6% | 0.1 |
| Mistral-7B | +4.4 | +4.4% | 0.0 |
| Llama-3.1-8B | +6.9 | +7.1% | 0.2 |
| Qwen2.5-14B† | +10.3 | +7.0% | — |

+1.46%, ours +4.41%). Qwen2.5-7B is the least robust under AWQ and our uniform RTN (+6.57% and +8.56%, respectively); under GPTQ, Llama-3.1-8B and Qwen2.5-7B are nearly tied (+4.37% vs. +4.38%, a 0.01 pp gap within measurement noise), so the full three-way ordering is not universal across methods. Mistral's advantage over Qwen2.5-7B, however, holds robustly across every quantization method, hardware platform, and software stack measured in this paper — the strongest evidence here that the architecture effect is a property of the models themselves, not an artifact of our specific RTN pipeline: it survives a change of quantization algorithm entirely, not merely a change of hardware (Table 4).

We flag the excluded Qwen2.5-14B GPTQ-Int4 point in the same spirit as the mixed-precision anomaly in Section 4.1: a result extreme enough to demand verification before being cited, which we performed (direct greedy generation from the loaded checkpoint produced repetitive, semantically incoherent text, confirming the model itself — not just our eval loop — was malfunctioning under this loading path), rather than a data point we are comfortable reporting as a genuine quantization-quality measurement.

## 4.3 Reproducibility Benchmark

**Setup.** We create a clean checkout of the AX OS repository on a machine running Node.js 22.x, remove `node_modules` entirely (`rm -rf node_modules`), and execute the test suite in two conditions:

1. *Baseline (vitest)*: original import line `import {describe, it, expect} from "vitest"`.
2. *Harness (`_harness.mjs`)*: updated import `import {describe, it, expect} from "./_harness.mjs"`.

No `npm install` is run in either condition.

Table 5: Paired bootstrap 95% CI (10,000 resamples of window indices, same indices applied to both conditions) for the CUDA ΔPPL% figures in Table 4. $n$ is the WikiText-2 window count.

| Model | $n$ | $\Delta\text{PPL}_{\text{CUDA}}$ | 95% CI (pp) |
|---|---|---|---|
| Qwen2.5-1.5B | 585 | +15.12 | [+14.80, +15.45] |
| Qwen2.5-3B | 585 | +11.96 | [+11.62, +12.31] |
| Qwen2.5-7B | 585 | +8.56 | [+8.28, +8.83] |
| Mistral-7B | 654 | +4.41 | [+4.27, +4.56] |
| Llama-3.1-8B | 565 | +7.11 | [+6.85, +7.38] |
| Qwen2.5-14B (NF4) | 585 | +6.99 | [+6.42, +7.57] |

The test runner is invoked as `node -test ax-os-tests/*.test.mjs`.

**Results.** Table 7 summarises the spec counts. In the baseline condition, Node.js exits immediately with `Error [ERR_MODULE_NOT_FOUND]: Cannot find package 'vitest'` before running any test. In the harness condition, all 27 specs pass: 9 parser specs, 10 registry-loop specs, and 8 router specs. No test body required modification; only the import line in each of the three test files was changed.

## 4.4 Oracle Switching: Live Validation

The oracle switching contribution couples a validated agent model to an external simulator via a single environment-variable gate (`BRAIN_REAL=1`). We evaluate the design on two dimensions: mock-mode overhead and live-path end-to-end connectivity.

**Setup.** The inline `realSim()` function requires no compiled `dist/` artifact: activation is controlled by `BRAIN_REAL` with negligible overhead in mock mode (one string equality check before any subprocess fork). For the live path, we submitted a test alpha expression to the WorldQuant BRAIN simulation API[2] from a clean Python 3 environment with no framework dependencies beyond the standard `requests` library. The subprocess timeout is set to 1,800,000 ms to accommodate the architecture-derived ∼140 s API round-trip.

[2]WorldQuant BRAIN is a quantitative investment simulation platform (`https://platform.worldquant.com`); the live oracle submits a trading strategy and waits for a backtested performance score.

Table 6: Our uniform q4 (RTN, group size 64) versus published AWQ and GPTQ-Int4 checkpoints, same six models, identical WikiText-2 protocol (full test split, 512-token non-overlapping windows). AWQ/GPTQ checkpoints are official Qwen releases (four Qwen sizes) or established community quantizations (Mistral: TechxGenus; Llama: hugging-quants), at group size 128 (vs. our 64) — each method's standard shipped configuration. †Qwen2.5-14B: our column is NF4, a different scheme (Table 4), not directly comparable. ‡Qwen2.5-14B GPTQ-Int4 is excluded: the checkpoint produced incoherent generations under direct sampling (verified manually) and PPL > 9× baseline on this hardware/kernel combination, indicating a loading or kernel defect rather than a genuine quantization-quality result; we did not have the opportunity to root-cause it further.

| Model | Ours (q4, g64) | AWQ | GPTQ-Int4 |
|---|---|---|---|
| Qwen2.5-1.5B | +15.12 | +8.96 | +12.89 |
| Qwen2.5-3B | +11.96 | +7.55 | +8.73 |
| Qwen2.5-7B | +8.56 | +6.57 | +4.38 |
| Mistral-7B | +4.41 | +2.51 | +1.46 |
| Llama-3.1-8B | +7.11 | +4.25 | +4.37 |
| Qwen2.5-14B† | n/a (NF4) | +9.30 | —‡ |

**Results.** Table 8 summarises the two-mode comparison. In mock mode, the gate short-circuits immediately: elapsed time is sub-millisecond and no subprocess is spawned. In live mode, the BRAIN API responded after 141.9 s and returned `alpha_id` RRd5Rvmj, confirming end-to-end connectivity. The measured latency is consistent with the architecture-derived estimate and with the ∼140 s round-trip reported in the WorldQuant BRAIN documentation for simulation jobs of this class. The one-line JSON protocol (`{"alpha_id": "...", "elapsed": ...}`) is independent of oracle internals and generalises to any Python-backed JSON-serialisable oracle.

A single live-mode run suffices for the intended claim: that the gate activates, the subprocess completes, and the response parses correctly. This is a design-correctness validation, not a latency charac-

Table 7: Spec counts by module for the installation-free harness. All 27 specs pass on a clean Node.js ≥18 without `npm install`.

| Test file | Specs | Dependencies |
|---|---|---|
| `parser.test.mjs` | 9 | none |
| `registry-loop.test.mjs` | 10 | none |
| `router.test.mjs` | 8 | none |
| **Total** | **27** | **none** |

Table 8: Oracle gate two-mode comparison. Mock mode incurs negligible overhead; live mode confirms end-to-end API connectivity in a single validation run.

| Mode | Elapsed | Result |
|---|---|---|
| Mock (`BRAIN_REAL=0`) | <1 ms | short-circuit, no subprocess |
| Live (`BRAIN_REAL=1`) | 141.9 s | `alpha_id` RRd5Rvmj |

terisation; repeated measurements under controlled conditions would be needed for the latter, which is outside the scope of this protocol demonstration. Reproduction requires a WorldQuant BRAIN account with simulation access; the mock mode (`BRAIN_REAL=0`) is fully self-contained and exercises all code paths short of the API call itself.

## 4.5 Cross-corpus Generalization

**Motivation.** The WikiText-2 findings establish ΔPPL sensitivity as a function of model scale and architecture on a curated, encyclopaedic corpus. A natural question is whether the observed patterns are corpus-specific artefacts or reflect a more fundamental property of q4 quantization on these architectures. To address this, we repeat the PPL evaluation on a structurally different corpus: C4 (Colossal Clean Crawled Corpus) [Raffel et al., 2020], a web-crawled English dataset that differs from WikiText-2 in register (informal web text vs. edited prose), length distribution (shorter, noisier documents vs. article-length text), and domain coverage (broad web vs. Wikipedia).

**Setup.** We use the first 200 documents of the C4 English validation split (streaming extraction, fixed prefix, ≈80K tokens[3]). The evaluation protocol is identical to Section 4.1: non-overlapping 512-token windows, each model's native tokeniser, stride 512, sequence length 512. We measure both BF16 and q4 ΔPPL directly for all four models (Qwen2.5-1.5B, Qwen2.5-3B, Qwen2.5-7B, Mistral-7B-Instruct-v0.3), using the same locally-converted artifacts as the WikiText-2 evaluation.

Table 9: Cross-corpus ΔPPL comparison for fully-paired models. C4 English validation (200 docs, ≈80K tokens) vs. WikiText-2 test (full). Absolute PPL values are not directly comparable across corpora; the ΔPPL ratio is corpus-independent for deterministic RTN q4. All models evaluated with matched BF16 and q4 artifacts under the identical 512-token window protocol.

| | C4 | | WikiText-2 | | ΔPPL | |
|---|---|---|---|---|---|---|
| Model | BF16 | q4 | BF16 | q4 | C4 | Wiki |
| Qwen2.5-1.5B | 18.11 | 21.04 | 12.70 | 14.60 | +16.2% | +15.0% |
| Qwen2.5-3B | 16.29 | 18.19 | 11.45 | 12.79 | +11.7% | +11.7% |
| Qwen2.5-7B | 14.77 | 15.92 | 10.14 | 11.01 | +7.80% | +8.5% |
| Mistral-7B | 10.08 | 10.53 | 7.24 | 7.56 | +4.47% | +4.4% |

**Results.** Table 9 presents the cross-corpus comparison for all four fully paired models. Absolute C4 PPL is higher than WikiText-2 for all variants—consistent with C4's noisier, more heterogeneous text—but ΔPPL on C4 closely tracks the WikiText-2 values.

The ΔPPL values for all four models (+16.2% for 1.5B, +11.7% for 3B, +7.80% for Qwen-7B, +4.47% for Mistral-7B on C4 vs. +15.0%, +11.7%, +8.5%, +4.4% on WikiText-2 respectively) establish that both the intra-family scaling trend—smaller Qwen2.5 models incur proportionally larger q4 degradation—and the cross-architecture gap at 7–8B—Mistral degrading substantially less than Qwen2.5-7B—hold across both corpora. The close correspondence of ΔPPL values between corpora (within 1.2pp for every model) supports the characterization of WikiText-2 ΔPPL

[3] C4 yields ≈400 tokens/document on average with the Qwen tokeniser (89,180 tokens with the Mistral tokeniser); 200 documents span approximately one-third of the WikiText-2 test set in token count.

as a robust, corpus-independent signal rather than a corpus-specific artefact.

As a cross-check, the absolute C4/WikiText-2 q4 PPL ratio is consistent within the Qwen2.5 family across three scales: 1.44× (1.5B), 1.42× (3B), and 1.45× (7B)—a spread of under 2%. The cross-architecture counterpart Mistral-7B yields a C4/WikiText-2 ratio of 1.39×, moderately lower than the Qwen-7B value (1.45×), consistent with Mistral's overall lower absolute PPL on both corpora.

# 5 Results and Discussion

## 5.1 Key Findings

**Scale-dependent q4 tolerance (Contribution 1, primary).** WikiText-2 standard evaluation reveals two distinct effects. First, within the Qwen2.5 model family, uniform q4 ΔPPL decreases from 1.5B to 7B (+15.0%, +11.7%, +8.5%); the 14B result (+10.3%) is slightly above 7B, indicating the sub-7B decrease does not continue at 14B. The 1.8× spread between the 1.5B maximum and 7B minimum spans a 9.3× parameter range. With four scale points we report this as an observation, not a characterised trend. Second, at matched 7B scale, cross-architecture variation provides additional separation: Mistral-7B degrades by only +4.4%, a 1.9× gap versus Qwen2.5-7B at the same scale and compression ratio. These results suggest that, in the 1.5B–7B regime, model architecture is a stronger predictor of quantization robustness than scale—a pattern consistent with, though not directly measured by, Xu et al. [2024b] and Ouyang et al. [2024] in the 7B–70B range.

One candidate mechanism for this cross-architecture difference is vocabulary sparsity. Table 10 presents the results of tokenising the complete WikiText-2 test split with each model's native tokeniser; three confounders (vocabulary size, architecture depth, training data) vary simultaneously, so this analysis is hypothesis-generating rather than causal.

Vocabulary utilisation rank-orders ΔPPL sensitivity across all three architectures: 12.3% → 15.0% → 41.8% corresponds to +8.5% → +6.9% → +4.4%.

Table 10: Vocabulary utilisation on WikiText-2 test split. Vocabulary utilisation (% of vocab seen) and hapax rate (% of active tokens appearing exactly once) are measured with each model's native tokeniser. Rank order of vocabulary utilisation matches rank order of ΔPPL sensitivity across all three architectures, consistent with a vocabulary-sparsity hypothesis (n=3; three confounders vary simultaneously).

| Model | Vocab | Util. (%) | Hapax (%) | ΔPPL |
|---|---|---|---|---|
| Qwen2.5-7B | 151,643 | 12.3 | 34.6 | +8.5% |
| Llama-3.1-8B | 128,000 | 15.0 | 34.7 | +6.9% |
| Mistral-7B | 32,768 | 41.8 | 22.3 | +4.4% |

Notably, Qwen2.5-7B and Llama-3.1-8B have nearly identical hapax rates (34.6% vs. 34.7%) yet differ in ΔPPL (+8.5% vs. +6.9%), indicating that hapax rate alone does not fully account for the gap; the lower vocabulary utilisation of Qwen2.5-7B (12.3% vs. 15.0%) may be the additional separating factor. One candidate explanation is that hapax tokens provide little averaging signal: a quantization error in a single-occurrence token's embedding row would propagate to only one NLL term rather than being diluted across many occurrences. We did not measure per-token error magnitudes directly, so this remains a plausible mechanism rather than a demonstrated one, and the correlation itself rests on only three models with three confounders (vocabulary size, architecture depth, training data) varying simultaneously. The Mistral-7B advantage (+4.4%) is associated with both its much lower hapax rate (22.3%) and its 3.4× higher vocabulary utilisation, though as noted above this is correlational, not a controlled ablation. The residual gap between Qwen2.5-7B and Llama-3.1-8B is consistent with absolute embedding-table scale: Qwen's table (151,643 × 3584 parameters, ≈2.2 B weights) is 4.1× larger than Mistral's (32,768 × 4096, ≈0.5 B) and 1.4× larger than Llama's (128,000 × 4096, ≈1.6 B), contributing proportionally more total quantization error. All three models use Grouped-Query Attention, so attention head count is not the primary driver. The three-way rank consistency supports the vocabulary-sparsity hypothesis; a controlled ablation varying vo-

Table 11: ARC-Easy accuracy for Qwen2.5-7B (NLL-per-token scoring, no chat template, $n = 2{,}376$, 95% CI ±2.0pp per condition, *not* a paired BF16–q4 interval). The +0.50pp difference is within the per-condition interval (no detectable degradation); the test is underpowered to rule out small effects.

| Variant | Accuracy (%) | ΔACC (pp) |
|---|---|---|
| Qwen2.5-7B BF16 | 52.53 | — |
| Qwen2.5-7B q4 | 53.03 | +0.50 |
| 95% CI | ±2.0 pp | |

Table 12: HellaSwag accuracy, 400 random validation examples (seed 42, NLL-per-token scoring, no chat template). 95% CI ≈ ±4.0 pp per condition (wider than Table 11's due to the smaller sample; not a paired interval, same caveat as Table 11).

| Variant | Accuracy (%) | ΔACC (pp) |
|---|---|---|
| Qwen2.5-7B BF16 | 78.25 | — |
| Qwen2.5-7B q4 | 78.25 | 0.00 |
| Mistral-7B BF16 | 79.50 | — |
| Mistral-7B q4 | 79.00 | −0.50 |

cabulary while holding architecture fixed would be needed to confirm causality.

**Downstream accuracy.** To confirm that the +8.5% WikiText-2 PPL elevation does not translate to task degradation, we apply NLL-per-token scoring (no chat template) to both Qwen2.5-7B variants on ARC-Easy [Clark et al., 2018] ($n = 2{,}376$; 95% CI ±2.0pp). Table 11 presents the results. The q4 model scores +0.50pp above BF16—well within the confidence interval—indicating no detectable accuracy change at 7B scale. With CI ±2.0pp, however, the evaluation is underpowered to rule out small degradation effects; the result should be interpreted as absence of evidence rather than evidence of absence. Note also that this ±2.0 pp interval is a per-condition Wald interval on each variant's own accuracy, not a paired confidence interval on the BF16–q4 *difference*; per-example predictions were not logged in this run, so a proper paired test (bootstrap or McNemar over matched examples) could not be computed retroactively and remains future work.

**Second downstream benchmark (HellaSwag).** To extend downstream coverage beyond a single model and a single task, we additionally score Qwen2.5-7B-Instruct and Mistral-7B-Instruct-v0.3 on 400 random HellaSwag [Zellers et al., 2019] validation examples (NLL-per-token scoring, no chat template, seed 42; Table 12). Qwen2.5-7B shows *no* measurable change (78.25% for both variants); Mistral-7B drops 0.50 pp (79.50% → 79.00%). Both differences sit far inside the ≈ ±4.0 pp per-condition 95% CI at this sample size — wider than ARC-Easy's because HellaSwag was scored on a 400-example subsample rather than the full validation split. As with ARC-Easy above, this is a per-condition interval, not a paired one, and we did not extend this benchmark to the remaining four models or log per-example predictions for a paired test; this result supports absence of evidence for degradation on two models and two tasks, not evidence of its absence across the full scale sweep.

**Cross-corpus consistency (C4).** The ΔPPL patterns observed on WikiText-2 generalise to C4 (web-crawled text), now confirmed for all four models with paired BF16+q4 measurements. For Qwen2.5-1.5B, C4 yields ΔPPL of +16.2% vs. +15.0% on WikiText-2; for Qwen2.5-3B, +11.7% vs. +11.7%; for Qwen2.5-7B, +7.80% vs. +8.5%; for Mistral-7B, +4.47% vs. +4.4%. Both the intra-family monotone ordering and the cross-architecture gap at 7–8B are preserved, and absolute deviations are within 1.2 pp across the two corpora for every model. This corpus-independence strengthens the interpretation of WikiText-2 ΔPPL as a model-intrinsic property rather than an evaluation artefact.

**Mixed-precision failure.** For the 1.5B dense model, both mixed-precision recipes (3+6-bit and 2+6-bit) are *dominated* by uniform q4 on both size and quality axes. This result challenges the intuition that "less important layers can use fewer bits" for small dense models: the precision-sensitive layers at 1.5B scale appear distributed throughout the network, so that uniform quantization incurs less total rounding error than concentrated low-bit assignments under this configuration. These recipes assign bit-width

purely by layer depth and are not activation-aware or outlier-aware, so they are considerably cruder than established mixed-precision methods such as AWQ [Lin et al., 2023] (protects the top 1% of salient weights by activation magnitude) or SqueezeLLM [Kim et al., 2024] (isolates outliers into a sparse full-precision component); this result speaks only to a naive depth-based split, not to whether principled salience-aware mixed-precision would also underperform uniform q4. The 2+6-bit recipe's perplexity (>160,000, a >2700× increase over bf16) is extreme enough that we cannot rule out a numerical or implementation issue compounding the genuine quantization difficulty; we did not have the opportunity to debug this configuration further, so it should be read as a data point warranting caution rather than a clean characterisation of mixed-precision degradation. Whether this holds across other small-scale architectures requires broader experimental validation.

**Reproducibility (Contribution 2, enabling).** The installation-free harness achieves 27/27 test pass with zero install footprint and no test-body modifications, whereas the vitest-import baseline fails immediately (zero tests executed, `MODULE_NOT_FOUND`) without a prior `npm install` (Section 4.3). The migration cost is one import line per test file. This result confirms that Node.js built-ins are sufficient for the full describe/it/expect test grammar used in the AX OS test suite.

**Oracle switching (Contribution 3).** The `BRAIN_REAL=1` gate adds negligible overhead in mock mode and generalises to any Python-backed JSON-serialisable oracle. The live path was executed once for end-to-end validation: the BRAIN API responded in 141.9 s and returned `alpha_id` RRd5Rvmj, confirming the oracle gate is operational (see Section 5.2).

**End-to-end coherence.** Together, these three findings address the complete edge-deployment decision: the quantization measurements establish *which model and precision to deploy*; the harness provides *how to validate that decision reproducibly on the target machine*; and the oracle design pattern establishes *how to couple the deployed agent to its external simulator*. The Mistral-7B artifact is registered in the `AEQIntegration` registry, validated by the zero-dependency harness, and ready to load as a drop-in agent model—demonstrating all three components operating as an integrated system.

## 5.2 Limitations and Scope

Several important caveats apply to the findings.

**Two hardware platforms; throughput untested on CUDA.** Perplexity measurements are now cross-validated on two independent stacks — Apple M1 Max/MLX and NVIDIA RTX 5070 Ti/CUDA (Table 4) — agreeing within 0.3 pp on every affine-q4 model. Compression ratios generalise further still (they reflect weight tensor sizes, which are hardware-independent). Throughput, however, was measured only on Apple M1 Max; CUDA throughput, and behaviour on other accelerators (AMD GPUs, mobile NPUs), remain untested.

**Four Qwen scale points and two cross-architecture references.** The intra-family finding (ΔPPL decreasing 1.5B to 7B, with the 14B result slightly above 7B) rests on four data points; measurements at 32B and above are needed to determine whether the sub-7B decrease resumes, plateaus, or the 14B uptick persists. The cross-architecture comparison involves two model pairs at the 7–8B scale: Mistral-7B (+4.4%) and Llama-3.1-8B (+6.9%) both outperform Qwen2.5-7B (+8.5%); other architectures (Phi, Gemma) remain uncharacterised. Extrapolating these trends below 1.5B or above 14B is a hypothesis, not an empirical result.

**Single artifact per configuration.** Each (model, precision) cell is a single quantized artifact evaluated once. Perplexity on a fixed corpus is deterministic given the artifact. MLX block-wise q4 quantization (RTN, group size 64) is a deterministic arithmetic operation on the BF16 weights: for each 64-weight block, `mx.quantize` computes the block's absmax, maps weights to the nearest 4-bit integer, and stores the result—no calibration data and no PRNG calls are involved (verified by inspecting `mlx_lm.utils` source: zero `random` or `seed` calls in the quantization path). WikiText-2 evaluation on a fixed artifact is likewise deterministic on a fixed hardware platform. The reported ΔPPL values are therefore exact measurements, not run-to-run estimates. The relevant residual uncertainty is cross-library variance

(e.g., GPTQ vs. MLX RTN at the same group size). An earlier version of this section compared our RTN numbers only against a literature proxy (Lin et al. 2023, Table 4: a different model family, LLaMA-2-7B, under a different eval protocol, 2048-token overlapping windows). Section 4.2 supersedes that proxy with a direct, matched-protocol comparison against real published AWQ and GPTQ checkpoints on the same six models used throughout this paper. The one residual confound there is group size (ours: 64; the published checkpoints: typically 128, each method's standard shipped configuration).

**Quantization pipeline (uniform full-corpus).** All six models in Table 2 are evaluated under an identical protocol: non-overlapping 512-token windows over the complete WikiText-2 test split, using each model's native tokenizer. All Qwen q4 models were requantized locally with `mlx_lm.convert` at group size 64 (eliminating the prior mlx-community confound). Mistral-7B BF16 and q4 were evaluated under the same protocol. This uniform methodology enables a direct, protocol-matched cross-architecture comparison with no short-corpus caveats or inter-protocol uncertainty. The additional C4 evaluation (Section 4.5) extends coverage to a second corpus; the BF16+q4 pair is fully measured for all four models (Qwen2.5-1.5B, Qwen2.5-3B, Qwen2.5-7B, and Mistral-7B).

**MoE models untested.** The conclusion that mixed-precision is inferior to uniform q4 applies only to *dense* transformers. Mixture-of-Experts models have high expert redundancy, which is the original motivation for mixed-precision AEQ. This hypothesis remains untested.

**BRAIN live simulation.** The live oracle path (`BRAIN_REAL=1`) was executed once for validation: a test alpha expression was submitted to the WorldQuant BRAIN simulation API, which responded with a backtested performance record (`alpha_id`: RRd5Rvmj) after a measured latency of 141.9 s—consistent with the architecture-derived estimate. A single validation run was performed to confirm end-to-end connectivity; broader oracle coupling experiments remain future work.

# 6 Conclusion

We presented AX OS as an integrated solution to three interdependent edge-deployment problems: selecting a model and precision level that retains acceptable quality under memory constraints; validating that selection reproducibly on the deployment machine without a build environment; and coupling the validated agent to an external simulator without compiled middleware. Addressing all three jointly, rather than in isolation, is the central contribution of this work.

The primary empirical finding is that, on WikiText-2 standard evaluation across six models (four Qwen2.5 sizes, Mistral-7B, and Llama-3.1-8B)—and confirmed on C4 web-crawled text for four of those models (Section 4.5)—uniform q4 $\Delta$PPL within the Qwen family decreases from 1.5B to 7B (+15.0%, +11.7%, +8.5%) at 3.5× compression (the 14B result is slightly above 7B; see Section 5.2). At the 7–8B scale, cross-architecture variation provides additional separation: Mistral-7B (+4.4%) and Llama-3.1-8B (+6.9%) are both more robust than Qwen2.5-7B (+8.5%), spanning a 1.9× range and motivating architecture-aware rather than scale-only quantization scheme selection.

To validate these measurements reproducibly on the deployment machine itself, we contribute two infrastructure components. The built-in-only `_harness.mjs` eliminates the vitest install requirement with a 40-line wrapper around Node.js built-ins, achieving 27/27 test pass—including the `registry-loop` tests that validate the compressed-model entries—on clean checkouts. The inline `BRAIN_REAL` oracle switching pattern couples a live Python simulation oracle to the agent demonstration script without requiring a compiled `dist/` directory, preserving portability across deployment platforms.

The Mistral-7B-Instruct-v0.3 q4 artifact (4,078 MB, ppl +4.4%, 35.6 tok/s on Apple M1 Max) is registered in the AX OS `AEQIntegration` registry as a drop-in agent model for consumer hardware with $\geq$5 GB free VRAM.

Future work includes extending the scale study to 32B and above parameter checkpoints to determine whether the sub-7B decreasing trend resumes, or the 14B uptick (+10.3% versus +8.5% at 7B) persists, validating mixed-precision AEQ on MoE architec-

tures, and measuring throughput on CUDA targets to complement the Apple Silicon results. Having controlled for the quantization-pipeline confound (Section 5.2), both the intra-family scale trend and the cross-architecture gap now rest on a uniform full-corpus comparison.

## Acknowledgements

The authors thank the MLX open-source community for the Apple Silicon inference and quantization tooling used in this work. No competing interests to declare.

## References

Oluwaseun A. Ajayi and Ogundepo Odunayo. Benchmarking on-device machine learning on Apple Silicon with MLX. arXiv preprint arXiv:2510.18921, 2025.

Wayner Barrios. Native LLM and MLLM inference at scale on Apple Silicon. In *Proceedings of the ACM EuroMLSys Workshop on Machine Learning Systems (EuroMLSys '26)*, 2026. arXiv:2601.19139.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? Try ARC, the AI2 reasoning challenge. *arXiv*, 2018.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. In *ICLR 2023*, 2023.

Odd Erik Gundersen and Sigbjoern Kjensmo. State of the art: Reproducibility in artificial intelligence. In *AAAI 2018*, 2018.

Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W. Mahoney, and Kurt Keutzer. Squeezellm: Dense-and-sparse quantization. In *ICML 2024*, 2024.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration. *MLSys 2024*, 2023. doi: 10.1145/3714983.3714987.

Yushen Lin, Sicheng Li, Zhenwu Peng, Jianlei Yang, and Wei Zhao. A review on edge large language models: Design, execution, and applications. arXiv preprint arXiv:2410.11845, 2024.

Xu Ouyang, Tao Ge, Thomas Hartvigsen, Zhisong Zhang, Haitao Mi, and Dong Yu. Low-bit quantization favors undertrained llms: Scaling laws for quantized llms with 100t training tokens. *arXiv*, 2024.

Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Lariviere, Alina Beygelzimer, Florence d'Alche Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (a report from the neurips 2019 reproducibility program). *Journal of Machine Learning Research*, 2021.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. arXiv preprint arXiv:1910.10683, 2020.

Varun Rajesh, Om Jodhpurkar, Pooja Anbuselvan, Mantinder Singh, Ashok Jallepali, Shantanu Godbole, Pradeep Kumar Sharma, and Hritvik Shrivastava. Production-grade local LLM inference on Apple Silicon: A comparative study of MLX, MLC-LLM, ollama, llama.cpp, and PyTorch MPS. arXiv preprint arXiv:2511.05502, 2025.

Zhongwei Wan, Xin Wang, Che Liu, Samiul Alam, Yu Zheng, Jiachen Liu, Zhongnan Qu, Shen Yan, Yi Zhu, Quanlu Zhang, Mosharaf Chowdhury, and Mi Zhang. Efficient large language models: A survey. *Transactions on Machine Learning Research*, 2024.

Jiajun Xu, Zhiyuan Li, Wei Wei, Guoxing Yang, Zhiwei Zeng, and Niraj Jha. On-device language models: A comprehensive review. arXiv preprint arXiv:2409.00088, 2024a.

Zifei Xu, Alexander Lan, Wanzin Yazar, Tristan Webb, Sayeh Sharify, and Xin Wang. Scaling laws for post training quantized large language models. *arXiv*, 2024b.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800, 2019.

Jiaqi Zhao, Ming Wang, Miao Zhang, Yuzhang Shang, Xuebo Liu, Yaowei Wang, Min Zhang, and Liqiang Nie. Benchmarking post-training quantization in llms: Comprehensive taxonomy, unified evaluation, and comparative analysis. *arXiv*, 2025.

Xunyu Zhu, Jian Li, Yong Liu, Can Ma, and Weiping Wang. A survey on model compression for large language models. *Transactions of the Association for Computational Linguistics*, 2024.